

Coding Solution

Date	11 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	4 Marks

1. Data Preprocessing (src/preprocessing.py)

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler

def preprocess(df):
    df = df.copy()
    # Convert multi-class payment status codes into binary risk labels
    df['payment_status'] = df['payment_status_code'].apply(
        lambda x: 1 if x in ['late', 'default', 'past_due'] else 0
    )
    # Encode categorical columns
    for col in ['employment_type', 'credit_history_flag']:
        df[col] = LabelEncoder().fit_transform(df[col])
    # Scale numeric columns
    num_cols = ['income', 'employment_duration', 'existing_loan_balance', 'credit_inquiries']
    scaler = StandardScaler()
    df[num_cols] = scaler.fit_transform(df[num_cols])
    return df, scaler
```

2. Model Training & Comparison (src/train_model.py)

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import pickle

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(n_estimators=200),
    "Decision Tree": DecisionTreeClassifier(),
    "XGBoost": XGBClassifier(use_label_encoder=False, eval_metric='logloss')
}

best_model, best_score = None, 0
for name, model in models.items():
    model.fit(X_train, y_train)
    score = accuracy_score(y_test, model.predict(X_test))
    print(f"{name}: {score:.4f}")
    if score > best_score:
        best_model, best_score = model, score

with open("models/best_model.pkl", "wb") as f:
    pickle.dump(best_model, f)
```

3. Flask Application (app.py)

```
from flask import Flask, render_template, request
import pickle
import numpy as np

app = Flask(__name__)
model = pickle.load(open("models/best_model.pkl", "rb"))

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():
    features = [float(x) for x in request.form.values()]
    prediction = model.predict([np.array(features)])[0]
    result = "Approved" if prediction == 1 else "Rejected"
    return render_template("result.html", prediction_text=f"Application Status: {result}")

if __name__ == "__main__":
    app.run(debug=True)
```

Summary

The solution is organized into three cooperating modules: a preprocessing module that produces consistent features for both training and inference, a

training script that compares four classification algorithms and persists the best model, and a Flask application that serves real-time predictions through a simple web form.

Project: Credit Card Approval Prediction